

```
print("--- Simple Output ---")

print("Hello, World!")
print("Welcome to Python!")
print("Learning is fun!")
```

```
--- Simple Output ---
Hello, World!
Welcome to Python!
Learning is fun!
```

```
print(42)           # Integer
print(3.14159)      # Float
print(True)         # Boolean
print(None)         # NoneType
print([1, 2, 3])    # List
print({"a": 1})     # Dictionary key - value pairs
```

```
42
3.14159
True
None
[1, 2, 3]
{'a': 1}
```

```
print(100)
print(5.5)
print(False)
print([10, 20, 30])
print({"name": "Kishore"})
```

```
100
5.5
False
[10, 20, 30]
{'name': 'Kishore'}
```

```
print("--- Multiple Items ---")
print("Hello", "World")
print("Python", "is", "awesome")
print("Name:", "Baraa", "| Age:", 25)
print(1, 2, 3, 4, 5)
```

```
--- Multiple Items ---
Hello World
Python is awesome
Name: Baraa | Age: 25
1 2 3 4 5
```

```
print("Loading", end="")
print("...", end="")
print(" Done!")

print("First line", end=" --> ")
print("Second line")
```

```
Loading... Done!
First line --> Second line
```

```
# Creating a progress bar effect
print("\nProgress: ", end="")
for i in range(5):
    print("█", end="")
print(" Complete!")
```

```
Progress: ██████ Complete!
```

```
import time

print("Downloading: ", end="")

for i in range(10):
    print("█", end="")
    time.sleep(0.5)

print(" Complete!")
```

Downloading: ██████████ Complete!

```
name = "Baraa"
age = 25
city = "Gaza"
is_student = False

print("Name:", name)
print("Age:", age)
print("City:", city)
print("Student:", is_student)
```

Name: Baraa
Age: 25
City: Gaza
Student: False

```
# Print with variable calculations
x = 10
y = 5
print("Sum:", x + y)
print("Product:", x * y)
```

Sum: 15
Product: 50

```
name = "Baraa"
age = 25
height = 1.75

print(f"My name is {name}")
print(f"I am {age} years old")
print(f"Next year I'll be {age + 1}")
```

My name is Baraa
I am 25 years old
Next year I'll be 26

```
# Formatting numbers
pi = 3.14159265359
print(f"Pi to 2 decimals: {pi:.2f}")
print(f"Pi to 4 decimals: {pi:.4f}")
```

Pi to 2 decimals: 3.14
Pi to 4 decimals: 3.1416

```
# Padding and alignment
print(f"{'Left':<10}|{'Center':^10}|{'Right':>10}")
print(f"{1:<10}|{2:^10}|{3:>10}")
```

Left	Center	Right
1	2	3

```
name = "Kishore"
dept = "AI"
mark = 95

print(f"{'Name':<15}|{'Dept':<10}|{'Mark':>5}")
print(f"{name:<15}|{dept:<10}|{mark:>5}")
```

Name	Dept	Mark
Kishore	AI	95

```
# Currency formatting
price = 1234.567
print(f"Price: ${price:,.2f}")
```

Price: \$1,234.57

```
revenue = 12500000.789

print(f"Revenue: ${revenue:,.2f}")
```

Revenue: \$12,500,000.79

```
print("\n--- Escape Characters ---")
```

```

print("Line 1\nLine 2\nLine 3")      # \n = newline
print("Column1\tColumn2\tColumn3")    # \t = tab
print("She said: \"Hello!\"")         # \" = quote
print('It\'s a beautiful day')        # \' = apostrophe
print("Path: C:\\Users\\Documents")    # \\ = backslash
print("Bell sound: \\a")               # \\a = bell (may not work)

# Raw strings - ignore escape characters
print("\\nRaw string:")
print(r"C:\\Users\\Baraa\\Desktop")    # r prefix for raw string

```

```

--- Escape Characters ---
Line 1
Line 2
Line 3
Column1 Column2 Column3
She said: "Hello!"
It's a beautiful day
Path: C:\Users\Documents
Bell sound: 🔔

Raw string:
C:\Users\Baraa\Desktop

```

```

# Using triple quotes
message = """
This is a multi-line message.
It spans across several lines.
Very useful for long text!
"""

print(message)

```

```

This is a multi-line message.
It spans across several lines.
Very useful for long text!

```

```

# ASCII art example
print("""
  _____
 |           |
 |  Welcome  |
 |  to Python!  |
 |  Let's    |
 |  learn    |
 |  together! |
 |           |
 |_____|_____|
""")

```

```

  _____
 |           |
 |  Welcome  |
 |  to Python!  |
 |  Let's    |
 |  learn    |
 |  together! |
 |           |
 |_____|_____|

```

```

# Receipt example
print("\n===== RECEIPT =====")
item1, price1 = "Coffee", 4.99
item2, price2 = "Sandwich", 8.50
item3, price3 = "Cookie", 2.25
total = price1 + price2 + price3

print(f"item1:<20>${price1:.2f}")
print(f"item2:<20>${price2:.2f}")
print(f"item3:<20>${price3:.2f}")
print("=" * 30)
print(f"{'TOTAL':<20>${total:.2f}")

```

```

===== RECEIPT =====
Coffee.....$4.99
Sandwich.....$8.50
Cookie.....$2.25
=====
TOTAL.....$15.74

```

```
# Table example
print("\n| Name      | Age | City      |")
print("|-----|-----|")
print(f"| {'Alice':<8} | {25:<3} | {'New York':<10} |")
print(f"| {'Bob':<8} | {30:<3} | {'London':<10} |")
print(f"| {'Charlie':<8} | {35:<3} | {'Tokyo':<10} |")
```

Name	Age	City
Alice	25	New York
Bob	30	London
Charlie	35	Tokyo

```
# Docstrings are special strings used to document modules, functions, classes
def greet(name):
    """
    Greet a person by their name.
    This function takes a person's name and prints a friendly
    greeting message to the console.
    Args:
        name (str): The name of the person to greet.
    Returns:
        str: A greeting message.
    Examples:
        >>> greet("Baraa")
        'Hello, Baraa! Welcome!'
    """
    return f"Hello, {name}! Welcome!"
# Access the docstring
print(greet("Baraa"))
print(f"Function docstring: {greet.__doc__[:50]}...")
```

```
Hello, Baraa! Welcome!
Function docstring:
    Greet a person by their name.
    This functi...
```

```
def calculate_area(length, width):
    """
    Calculate the area of a rectangle.

    Args:
        length (float): The length of the rectangle.
        width (float): The width of the rectangle.

    Returns:
        float: The area of the rectangle.

    Raises:
        ValueError: If length or width is negative.
    """
    if length < 0 or width < 0:
        raise ValueError("Length and width must be non-negative")
    return length * width
```

```
print(f"Area: {calculate_area(5, 3)}")
```

```
Area: 15
```

```
class Rectangle:
    """
    A class to represent a rectangle.

    This class provides methods to calculate the area and perimeter
    of a rectangle, as well as other utility methods.

    Attributes:
        length (float): The length of the rectangle.
        width (float): The width of the rectangle.

    Methods:
        area(): Returns the area of the rectangle.
        perimeter(): Returns the perimeter of the rectangle.
    """

    def __init__(self, length, width):
        """
        Initialize a Rectangle instance.
```

```

    Args:
        length (float): The length of the rectangle.
        width (float): The width of the rectangle.
    """
    self.length = length
    self.width = width

    def area(self):
        """Calculate and return the area of the rectangle."""
        return self.length * self.width

    def perimeter(self):
        """Calculate and return the perimeter of the rectangle."""
        return 2 * (self.length + self.width)

rect = Rectangle(10, 5)
print(f"Rectangle area: {rect.area()}")
print(f"Rectangle perimeter: {rect.perimeter()}")

```

```

Rectangle area: 50
Rectangle perimeter: 30

```

```

print("\n--- Commenting Out Code ---")

# You can temporarily disable code by commenting it out
# print("This line won't execute")
# old_function()
# deprecated_code()

# Useful during debugging
value = 100
# value = 200 # Uncomment to test with different value
print(f"Current value: {value}")

# Multiple lines can be commented at once
# line_1 = "first"
# line_2 = "second"
# line_3 = "third"

```

```

--- Commenting Out Code ---
Current value: 100

```

```

# Simple variable assignments
name = "Baraa"
age = 25
height = 1.75
is_student = True

print(f"Name: {name}")
print(f"Age: {age}")
print(f"Height: {height}m")
print(f"Is student: {is_student}")

```

```

Name: Baraa
Age: 25
Height: 1.75m
Is student: True

```

```

# Variables can be reassigned to different values
x = 10
print(f"\nx = {x}")
x = 20
print(f"x = {x} (reassigned)")

```

```

x = 10
x = 20 (reassigned)

```

```

# Variables can even change types (dynamic typing)
value = 100
print(f"\nvalue = {value} (type: {type(value).__name__})")
value = "one hundred"
print(f"value = {value} (type: {type(value).__name__})")

```

```

value = 100 (type: int)
value = one hundred (type: str)

```

```
# Valid variable names
my_variable = 1      # Lowercase with underscores (snake_case) - RECOMMENDED
myVariable = 2       # camelCase - valid but not Pythonic
MyVariable = 3       # PascalCase - usually for classes
_private_var = 4     # Starts with underscore - convention for private
__very_private = 5   # Double underscore - name mangling
var123 = 6           # Can contain numbers (but not start with them)
CONSTANT = 7         # ALL CAPS - convention for constants

print(f"my_variable = {my_variable}")
print(f"_private_var = {_private_var}")
print(f"CONSTANT = {CONSTANT}")
my_variable = 1
_private_var = 4
CONSTANT = 7
```

```
# Reserved keywords in Python (cannot use as variable names):
import keyword
print(f"\nPython reserved keywords: {keyword.kwlist[:10]}...")
```

Python reserved keywords: ['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class']...

```
# Variables and functions: snake_case
user_name = "john_doe"
max_value = 100
is_active = True

# Constants: UPPER_SNAKE_CASE
PI = 3.14159
MAX_CONNECTIONS = 100
DEFAULT_TIMEOUT = 30

# Classes: PascalCase (CapWords)
# class MyClass:
#     pass

# Private variables: leading underscore
_internal_state = "private"

# "Very private" / Name-mangled: double leading underscore
__name_mangled = "mangled"

print(f"user_name: {user_name}")
print(f"PI (constant): {PI}")
print(f"MAX_CONNECTIONS (constant): {MAX_CONNECTIONS}")
```

```
user_name: john_doe
PI (constant): 3.14159
MAX_CONNECTIONS (constant): 100
```

```
# Assign the same value to multiple variables
a = b = c = 100
print(f"a = {a}, b = {b}, c = {c}")
```

```
a = 100, b = 100, c = 100
```

```
# Assign different values in one line
x, y, z = 1, 2, 3
print(f"x = {x}, y = {y}, z = {z}")
```

```
x = 1, y = 2, z = 3
```

Start coding or [generate](#) with AI.